Chapter 7. Convolutional Neural Networks and Architectures

Convolution, stride, padding, pooling, equivariance; AlexNet, LRN, ZFNet, VGG, GoogLeNet/Inception, ResNet, saliency maps

Contents

1 Why a New Layer **2**

2 Convolution as a Local Dot Product **2**

2.1 A fixed filter: moving average 2

2.2 Single and multiple channels 2

2.3 Parameter counts 3

3 Stride, Padding and Output Size **3**

4 Learning Filters **4**

5 Shift Equivariance **4**

6 1×1 Convolution, Pooling and Global Average Pooling **5**

7 Algebraic Facts about CNN Layers **5**

8 Vocabulary and Costs **6**

9 AlexNet **6**

10 ZFNet and VGG **6**

11 GoogLeNet and the Inception Module **6**

11.1 Naive module 7

11.2 Bottleneck module 7

11.3 Split–transform–merge 7

12 Why Depth Can Fail, and Residual Networks **7**

13 Saliency Maps **8**

1 Why a New Layer

An image $X \in \mathbb{R}^{H\times W\times C}$ fed to a fully connected layer is flattened, and $z = A\mathbf{x} + \mathbf{b}$ with $A \in \mathbb{R}^{M\times HWC}$ has $M(HWC+1)$ parameters: for $32\times 32\times 3$ input and $M = 1000$, already 3,073,000. Flattening loses no numbers, but an unrestricted matrix does not encode which pixels are neighbors or that a pattern can appear anywhere. Two assumptions about images motivate convolution:

- **Locality**: nearby pixels jointly form edges, corners and textures, so a unit can look at a small patch (local connectivity).
- **Stationarity**: the same pattern may occur at any position, so the same detector should be used everywhere (weight sharing).

Definition 7.1: equivariance and invariance

Let $T_{\boldsymbol{\tau}}$ shift an image, $(T_{\boldsymbol{\tau}}X)_{i,j,c} = X_{i-\tau_1,j-\tau_2,c}$. A feature extractor F is shift *equivariant* if $F(T_{\boldsymbol{\tau}}X) = T_{\boldsymbol{\tau}}F(X)$ (the output moves with the input) and a classifier G is shift *invariant* if $G(T_{\boldsymbol{\tau}}X) = G(X)$ (the output does not change). More generally, for a group acting by $\rho_{\text{in}}(g)$, $\rho_{\text{out}}(g)$, f is equivariant if $f(\rho_{\text{in}}(g)x) = \rho_{\text{out}}(g)f(x)$; invariance is the case $\rho_{\text{out}} = \text{id}$.

2 Convolution as a Local Dot Product

2.1 A fixed filter: moving average

With the 3×3 averaging kernel (1/9 everywhere), $Y_{i,j} = \frac{1}{9}\sum_{u,v=0}^{2} X_{i+u,j+v}$: on an image of 0s and 90s a window with r bright pixels outputs $10r$. Averaging suppresses isolated pixels but blurs edges; a 5×5 average smooths more. A CNN *learns* the kernel; learned weights may be negative and need not sum to one.

2.2 Single and multiple channels

Definition 7.2: convolutional layer

With D filters $W \in \mathbb{R}^{D\times K\times K\times C}$, biases $\mathbf{b} \in \mathbb{R}^D$, stride S, padding P,

$$Z_{i,j,d} = b_d + \sum_{u=0}^{K-1}\sum_{v=0}^{K-1}\sum_{c=1}^{C} W_{d,u,v,c}\,\tilde{X}_{Si+u,\,Sj+v,\,c},$$

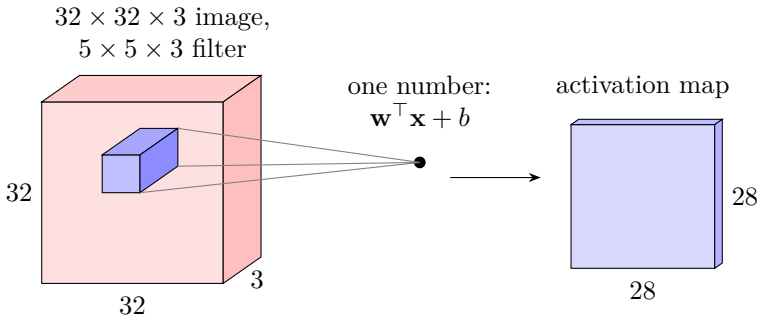
where $\tilde{X}$ is the padded input. Each output is a dot product of a $K\times K\times C$ patch with a filter spanning *all* input channels, plus a bias. The array $Z_{\cdot,\cdot,d}$ is the feature map of filter d; stacking the D maps gives D output channels. This operation is cross-correlation (no kernel flip); since kernels are learned, the distinction does not matter for expressiveness.

Example 7.3: one channel, 2×2 filter

$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}$, $W = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$, $b = 1$, $S = 1$, $P = 0$: $Z = \begin{pmatrix} 4 & 8 \\ 1 & 1 \end{pmatrix}$. The same four weights and bias are reused at the four positions.

Example 7.4: a 5×5 example

With a 5×5 image and $W = \begin{pmatrix} 1 & 2 & 3 \\ -4 & 7 & 4 \\ 2 & -5 & 1 \end{pmatrix}$ ($b = 0$): unpadded $Z = \begin{pmatrix} 51 & 66 & 20 \\ 31 & 49 & 101 \\ 15 & 53 & -2 \end{pmatrix}$; with $P = 1$ the output is 5×5 and its central 3×3 block equals the unpadded output (those windows contain the same pixels).



A filter spans the full depth of the input; sliding it over all spatial positions gives one 28×28 activation map.

2.3 Parameter counts

One $5 \times 5 \times 3$ filter has 75 weights and one bias; six filters on $32 \times 32 \times 3$ give a $28 \times 28 \times 6$ output with $6(75 + 1) = 456$ parameters. For the same input and output shape:

Layer	Parameters
Fully connected	$4704(3072 + 1) = 14{,}455{,}392$
Locally connected, unshared	$28^2 \cdot 6(75 + 1) = 357{,}504$
Convolution (shared)	$6(75 + 1) = 456$

Locality removes long-range connections; sharing removes independent copies. As a matrix acting on the flattened input, convolution is sparse with repeated entries (a 1D length-2 filter (a, b) on $(x_1, \dots, x_4)$ gives a banded matrix with only two free numbers).

3 Stride, Padding and Output Size

Motivation. Designing a CNN requires knowing the shape of every layer's output. A single formula relates input size, kernel size, padding and stride.

Theorem 7.5: output size

For input length N, kernel K, stride S, padding P (symmetric), the number of output positions is

$$N' = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1, \qquad C' = D.$$

Proof. The padded length is $N + 2P$. A window starting at tS ends at $tS + K - 1$, which must be $\le N + 2P - 1$, i.e. $t \le (N + 2P - K)/S$. The admissible integers are $t = 0, \dots, \lfloor (N + 2P - K)/S \rfloor$. ■

$N = 7$, $K = 3$: $S = 1 \Rightarrow 5$, $S = 2 \Rightarrow 3$, $S = 3 \Rightarrow 2$ (the last does not "fit" exactly; a border is left unused). For $S = 1$ and odd K, $P = (K-1)/2$ preserves the size (padding $1, 2, 3$ for $K = 3, 5, 7$). Zero padding is an explicit boundary assumption, not recovered information.

Example 7.6: further examples

Input $52 \times 52 \times 128$, $D = 256$.
(1) $K = 3, S = 1, P = 1$: output $52 \times 52 \times 256$, parameters $256(9 \cdot 128 + 1) = 295{,}168$; each filter is $3 \times 3 \times 128$.
(2) $K = 4, S = 2, P = 1$: $\lfloor (52 + 2 - 4)/2 \rfloor + 1 = 26$, output $26 \times 26 \times 256$, parameters $256(16 \cdot 128 + 1) = 524{,}544$. Parameters, activations (HWD) and multiply–accumulates ($\approx HWDK^2C$) are different costs.

Stacking: $32 \times 32 \times 3 \to 28 \times 28 \times 6 \to 24 \times 24 \times 10$ with ten $5 \times 5 \times 6$ filters ($10(150 + 1) = 1510$ parameters). Two stacked stride-1 5×5 layers see a 9×9 patch (receptive field $5 + 4$). With strides, $r_\ell = r_{\ell-1} + (K_\ell - 1) j_{\ell-1}$ and $j_\ell = S_\ell j_{\ell-1}$, starting from $r_0 = j_0 = 1$.

4 Learning Filters

For an unpadded stride-1 layer with upstream $G_{i,j,d} = \partial L / \partial Z_{i,j,d}$,

$$\frac{\partial L}{\partial W_{d,u,v,c}} = \sum_{i,j} G_{i,j,d} X_{i+u,j+v,c}, \qquad \frac{\partial L}{\partial b_d} = \sum_{i,j} G_{i,j,d} :$$

every position contributes to the update of the shared weight (the backward counterpart of weight sharing).

5 Shift Equivariance

Motivation. Shifting an image should shift the features, not change them. Convolution has exactly this property, which is the mathematical reason CNNs need far fewer parameters than fully connected networks on images.

Theorem 7.7: convolution commutes with shifts

On an infinite grid (or with circular indices), stride-1 convolution satisfies $C_W(T_{\boldsymbol{\tau}} X) = T_{\boldsymbol{\tau}} C_W(X)$, and pointwise nonlinearities also commute with shifts. Global average pooling of a circularly shifted map is unchanged, so GAP $\circ$ (convolutions, ReLUs) is shift invariant.

Proof. $[C_W(T_{\boldsymbol{\tau}} X)]_{i,j,d} = b_d + \sum_{u,v,c} W_{d,u,v,c} X_{i+u-\tau_1, j+v-\tau_2, c} = [C_W X]_{i-\tau_1, j-\tau_2, d}$; the coefficients do not depend on the absolute position (i, j). A circular shift permutes positions, and an average is invariant under permutations. ■

Remark 7.8: when exactness fails

With zero padding, content shifted across the border is lost and zeros enter, so equality holds only away from the boundary. A stride-S layer (or pooling) is compatible only with shifts that are multiples of S: for width-2, stride-2 max pooling M, $M(T_2 x) = T_1 M(x)$ but $M(T_1 x)$ need not be a shift of $M(x)$ (e.g. $x = (0, 1, 2, 0)$, $M(x) = (1, 2)$, $M(T_1 x) = (0, 2)$).

6 1×1 Convolution, Pooling and Global Average Pooling

1×1 **convolution.** At each position $\mathbf{y}_{i,j} = A\mathbf{x}_{i,j} + \mathbf{b}$ with $A \in \mathbb{R}^{D \times C}$ shared across positions: it mixes channels, not neighbors. $56 \times 56 \times 64 \to 56 \times 56 \times 32$ costs $32(64+1) = 2080$ parameters.

Pooling. Channelwise, no parameters: $Y^{\max}_{i,j,c} = \max_{u,v} X_{iS+u,jS+v,c}$ or the average. For $X = \begin{pmatrix} 5 & 10 & 6 & 7 \\ 4 & 0 & 4 & 3 \\ 2 & 4 & 3 & 4 \\ 0 & 8 & 1 & 6 \end{pmatrix}$ and 2×2, stride 2: average $\begin{pmatrix} 4.75 & 5 \\ 3.5 & 3.5 \end{pmatrix}$, max $\begin{pmatrix} 10 & 7 \\ 8 & 6 \end{pmatrix}$. Pooling reduces resolution and gives limited robustness to small shifts within a window. Backward: max pooling routes the gradient to the maximizer; average pooling spreads g/K^2.

Global average pooling. $g_c = \frac{1}{HW} \sum_{i,j} X_{i,j,c}$ maps $H \times W \times C \to C$; a classifier on top needs $Q(C+1)$ instead of $Q(HWC+1)$ parameters. It discards spatial arrangement entirely.

7 Algebraic Facts about CNN Layers

Motivation. The building blocks of CNNs (convolution, ReLU, pooling) satisfy simple algebraic identities. They tell us when layers can be reordered or merged without changing the network.

Theorem 7.9: positive homogeneity

A network built only from bias-free convolutions and linear layers, ReLU, max pooling, reshaping and dropout satisfies $f(\lambda X) = \lambda f(X)$ for all $\lambda \geq 0$.

Proof. Each component is positively homogeneous: linear maps for all λ; $\max(0, \lambda t) = \lambda \max(0, t)$ and $\max_i \lambda x_i = \lambda \max_i x_i$ for $\lambda \geq 0$ (scaling by $\lambda > 0$ preserves order and signs; $\lambda = 0$ is trivial); dropout multiplies by a fixed mask. Compositions of positively homogeneous maps are positively homogeneous: $g_1(g_2(\lambda X)) = g_1(\lambda g_2(X)) = \lambda g_1(g_2(X))$. Biases break the property, e.g. $\mathrm{ReLU}(x+1)$ at $x = 0$. ■

Theorem 7.10: conv–MP–ReLU orders coincide

conv→MaxPool→ReLU, conv→ReLU→MaxPool and conv→ReLU→MaxPool→ReLU are the same function.

Proof. For nondecreasing ϕ, $\max_i \phi(x_i) = \phi(\max_i x_i)$: $\phi(x_i) \leq \phi(\max x)$ for all i with equality at the maximizer. ReLU is nondecreasing and elementwise, and max pooling is a windowwise max, so $\mathrm{ReLU} \circ \mathrm{MP} = \mathrm{MP} \circ \mathrm{ReLU}$; finally $\mathrm{ReLU} \circ \mathrm{ReLU} = \mathrm{ReLU}$. (For average pooling only avg ReLU $\geq$ ReLU avg holds, by convexity.) ■

Theorem 7.11: convolutions that implement given maps

With PyTorch's convention $Y_{o,i,j} = b_o + \sum_c \sum_{k,l=1}^{3} w_{o,c,k,l} \tilde{X}_{c,i+k-2,j+l-2}$ (3×3, padding 1), reading the value at offset $(\Delta i, \Delta j)$ requires a 1 at $(k,l) = (2+\Delta i, 2+\Delta j)$.

- Identity $C \to C$: $w_{c,c',k,l} = \delta_{cc'}\delta_{k2}\delta_{l2}$, $\mathbf{b} = \mathbf{0}$.
- Colorwise finite differences $Y_{2c-1} = X_{c,i+1,j} - X_{c,i,j}$, $Y_{2c} = X_{c,i,j+1} - X_{c,i,j}$: $w_{2c-1,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$, $w_{2c,c} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 0 & 0 \end{pmatrix}$, all other slices zero; zero padding supplies the out-of-range zeros.

8 Vocabulary and Costs

A network maps an image to class scores and then probabilities, $X \mapsto h(X) \mapsto \mathbf{s}(X) \in \mathbb{R}^Q \mapsto \mathbf{p}(X)$. For a convolution with D filters $K \times K \times C$ producing $H' \times W' \times D$:

$$N_{\text{param}} = D(K^2C + 1), \qquad N_{\text{act}} = H'W'D, \qquad N_{\text{MAC}} \approx H'W'DK^2C.$$

An FC layer $\mathbb{R}^n \to \mathbb{R}^m$ has $m(n+1)$ parameters. Top-1 error asks whether the highest score is correct; top-5 error asks whether the true label is missing from the five highest scores.

9 AlexNet

Five convolutional and three FC layers (eight weight layers):

Stage	Settings	Output
Input	RGB	$227 \times 227 \times 3$
Conv1	96, 11×11, $S = 4$, $P = 0$	$55 \times 55 \times 96$ $\quad((227 - 11)/4 + 1)$
Pool1, Norm1	3×3, $S = 2$; LRN	$27 \times 27 \times 96$
Conv2	256, 5×5, $P = 2$	$27 \times 27 \times 256$
Pool2, Norm2	3×3, $S = 2$; LRN	$13 \times 13 \times 256$
Conv3, Conv4	384, 3×3, $P = 1$	$13 \times 13 \times 384$
Conv5	256, 3×3, $P = 1$	$13 \times 13 \times 256$
Pool3	3×3, $S = 2$	$6 \times 6 \times 256 = 9216$
FC6, FC7, FC8	$9216 \to 4096 \to 4096 \to 1000$	1000 scores

The FC layers dominate: $4096 \cdot 9217 = 37{,}752{,}832$, $4096 \cdot 4097 = 16{,}781{,}312$, $1000 \cdot 4097 = 4{,}097{,}000$, together 58,631,144 parameters, whereas Conv1 has $96(11^2 \cdot 3 + 1) = 34{,}944$. Design choices:

- **ReLU**: avoids sigmoid/tanh saturation on the positive side.
- **Large first stride**: $227 \to 55$ immediately reduces memory and computation (at the cost of detail).
- **Local response normalization**: $a_{i,j,c} \mapsto a_{i,j,c}/(k + \alpha \sum_{r \in N(c)} a_{i,j,r}^2)^\beta$ normalizes across neighboring channels (not batch normalization).
- **Dropout** 0.5 in the FC layers against overfitting.

10 ZFNet and VGG

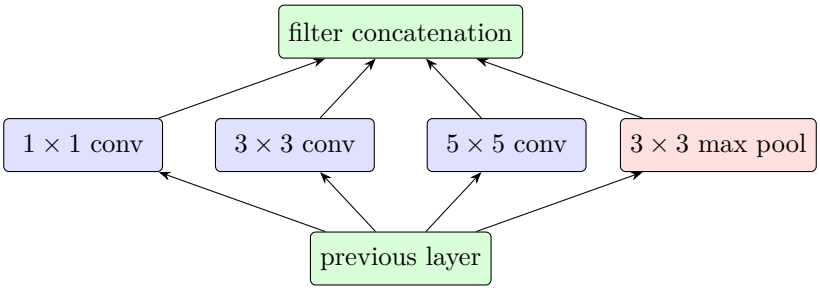
ZFNet tunes AlexNet's kernel sizes and strides. VGG uses deeper stacks of small 3×3 convolutions: two stride-1 3×3 layers have a 5×5 receptive field with $18C^2$ weights (versus $25C^2$), three have 7×7 with $27C^2$ (versus $49C^2$), and interleaved ReLUs add nonlinearity. Receptive fields obey $r_\ell = r_{\ell-1} + (K_\ell - 1)j_{\ell-1}$, $j_\ell = S_\ell j_{\ell-1}$.

11 GoogLeNet and the Inception Module

Motivation. Which filter size should a layer use? Inception uses several in parallel and concatenates the results, with 1×1 convolutions to keep the cost under control.

11.1 Naive module

For input $28 \times 28 \times 256$, parallel branches with padding chosen to keep 28×28: 1×1 (128), 3×3 (192), 5×5 (96), 3×3 max pool (stride 1). Channel concatenation gives $128 + 192 + 96 + 256 = 672$ channels. Weights: $256 \cdot 128 + 9 \cdot 256 \cdot 192 + 25 \cdot 256 \cdot 96 = 1{,}089{,}536$.



The naive Inception module: parallel convolutions of several sizes and a pooling branch, concatenated along the depth.

11.2 Bottleneck module

Insert 1×1 reductions to 64 channels before the 3×3 and 5×5 convolutions and after pooling: outputs $128 + 192 + 96 + 64 = 480$ channels and

$$32{,}768 + (16{,}384 + 110{,}592) + (16{,}384 + 153{,}600) + 16{,}384 = 346{,}112 \text{ weights}$$

(about 330k), a 68% reduction.

Theorem 7.12: when a bottleneck saves weights

A direct $K \times K$ map $C \to D$ costs K^2CD; reducing to R channels first costs $CR + K^2RD$. This is cheaper iff $R < \dfrac{K^2CD}{C + K^2D}$ (for $K = 5, C = 256, D = 96$ the threshold is ≈ 231).

The 1×1 convolution is a channel-mixing matrix applied at every position (see above). GoogLeNet stacks such modules (22 weight layers) and replaces the large FC stack by global average pooling plus a final classifier.

11.3 Split–transform–merge

A $256 \to 128 \to 128 \to 256$ conv–ReLU block (1×1, 3×3 with padding 1, 1×1, all with biases) has $32{,}896 + 147{,}584 + 33{,}024 = 213{,}504$ parameters. Thirty-two parallel $256 \to 4 \to 4 \to 256$ paths summed at the end have $32(1028 + 148 + 1280) = 78{,}592$ parameters; the 3×3 cost drops by the group factor 32. Implementation: three `nn.ModuleLists` of 32 convolutions, ReLU inside each path, then a sum (equivalently grouped convolutions with `groups=32`).

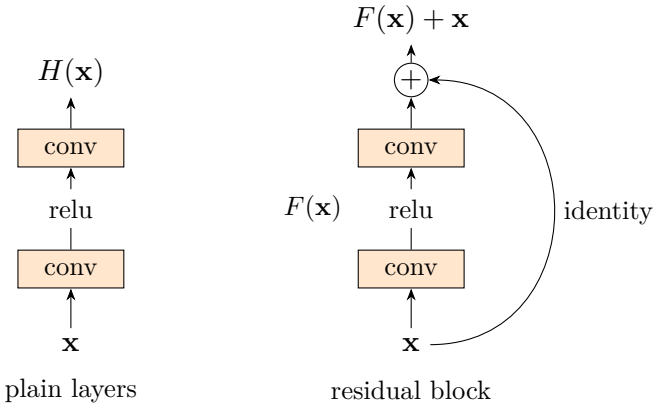
12 Why Depth Can Fail, and Residual Networks

Deeper plain networks can have *higher training error*: this is not overfitting but an optimization (degradation) problem. A deeper network that can represent identity layers contains the shallower one, so its *optimal* training loss is not worse, yet gradient methods may not find it. For $\mathbf{x}_{\ell+1} = f_\ell(\mathbf{x}_\ell)$, $\nabla_{\mathbf{x}_0} L = J_0^\top \cdots J_{L-1}^\top \nabla_{\mathbf{x}_L} L$; if every Jacobian has norm at most $\rho < 1$ the gradient shrinks like ρ^L ($0.9^{50} \approx 0.005$).

Theorem 7.13: residual block

$\mathbf{y} = \mathbf{x} + F(\mathbf{x};\boldsymbol{\theta})$ gives $\dfrac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + J_F(\mathbf{x})$ and $\nabla_{\mathbf{x}} L = \mathbf{g} + J_F^{\top}\mathbf{g}$ with $\mathbf{g} = \nabla_{\mathbf{y}} L$.

The identity path carries the gradient even when J_F is small, and the block can represent the identity by $F = 0$: the network learns a residual $F = H - \mathrm{id}$ rather than the whole map H. This is a reparametrization, not a guarantee (e.g. $J_F = -\mathbf{I}$ cancels the gradient; a ReLU after the sum alters the identity). If shapes differ, use a projection shortcut $\mathbf{y} = P(\mathbf{x}) + F(\mathbf{x})$ (e.g. a 1×1 or strided convolution). Unlike Inception's concatenation, residual addition keeps the channel count. A bottleneck block $256 \to 64 \to 64 \to 256$ (1×1, 3×3, 1×1) has $2 \cdot 256 \cdot 64 + 9 \cdot 64^2 = 69{,}632$ weights versus $18 \cdot 256^2 = 1{,}179{,}648$ for two full-width 3×3 layers. In the CIFAR-10 plots, residual networks remove the training-error degradation seen in plain networks. The full backpropagation formulas for a residual MLP are derived in Chapter 5.



A residual block learns $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$; with $F = 0$ it is the identity, and gradients pass through the shortcut.

13 Saliency Maps

To see which pixels a prediction is sensitive to, differentiate the selected class score with respect to the input (not the weights):

$$G_{i,j,k} = \frac{\partial s_c(X)}{\partial X_{i,j,k}}, \qquad M_{i,j} = \max_k |G_{i,j,k}|.$$

By Taylor's theorem $s_c(X + \Delta X) \approx s_c(X) + \langle \nabla_X s_c(X), \Delta X \rangle$: a large $|G|$ means a small change of that pixel changes the score a lot, and the sign says in which direction. For $s(\mathbf{x}) = 2x_1 - 3x_2 + 0.1x_3$, the gradient $(2, -3, 0.1)$ ranks x_2 first. Saliency is local sensitivity, not a complete causal explanation.